SET 15

PART A — Short questions and answers

Q1. Explain partial participation of MEDICINE in the relationship "Medicine is sold".

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Compare a super key and a candidate key using attributes of a MEDICINE table.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q3. Explain how functional dependencies help in finding a candidate key.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q4. Describe the use of ORDER BY ... DESC to list medicines by price.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Interpret the query: SELECT MedicineID, SUM(Quantity) FROM SALE GROUP BY MedicineID;

Answer: GROUP BY collects rows with the same value so aggregate functions can calculate a result for each group.

Q6. Describe the use of IF–ELSIF–ELSE in a PL/SQL block.

Answer: PL/SQL adds procedural programming to SQL. A procedure is a reusable named block that can take parameters.

Q7. Summarize the advantages of document databases over relational databases for a product catalogue.

Answer: Document databases allow documents to have different fields, making them useful when records vary.

Q8. Describe the ObjectId type.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Describe the output of the MongoDB query: db.medicines.find({ExpiryDate: {$lt: ISODate("2026-12-31")}})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Compare the MERGE and CREATE clauses in Cypher.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: A retail pharmacy maintains a Pharmacy Management System to store medicine details, customer

1(a) ER design:

• **MEDICINE**(MedicineID, MedicineName, Manufacturer, Price, ExpiryDate); primary key: **MedicineID**.

• **CUSTOMER**(CustomerID, CustomerName, Phone, City); primary key: **CustomerID**.

• **SALE**(SaleID, CustomerID, MedicineID, SaleDate, Quantity); primary key: **SaleID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE MEDICINE (
  MedicineID NUMBER PRIMARY KEY,
  MedicineName VARCHAR2(100),
  Manufacturer VARCHAR2(100),
  Price NUMBER,
  ExpiryDate DATE
);
CREATE TABLE CUSTOMER (
  CustomerID NUMBER PRIMARY KEY,
```

```
  CustomerName VARCHAR2(100),
  Phone VARCHAR2(100),
  City VARCHAR2(100)
);
CREATE TABLE SALE (
  SaleID NUMBER PRIMARY KEY,
  CustomerID NUMBER,
  MedicineID NUMBER,
  SaleDate DATE,
  Quantity NUMBER
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM SALE x
JOIN MEDICINE a ON x.MedicineID = a.MedicineID
JOIN CUSTOMER b ON x.CustomerID = b.CustomerID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. MongoDB: Create the collections and insert documents using the fields listed in the set. Query with find() and the requested filter.

```
db.collection.insertMany([
  { _id: 1, name: 'Sample A', category: 'Example', price: 500 },
  { _id: 2, name: 'Sample B', category: 'Example', price: 300 }
]);
db.collection.find({ author: 'Specified Author' });
db.collection.find({ price: { $gt: 400 } });
```